Exoplanet Transit Detection and Analysis using Lightkurve

Viraj Chaudhari

October 2026

1 Introduction

The search for exoplanets has advanced rapidly with space telescopes from missions like NASA's Transiting Exoplanet Survey Satellite (TESS). By constantly monitoring the stellar brightness, TESS captures the periodic drops in flux caused by transiting planets. However, processing these light curves requires filtering to eliminate instrumental noise, cosmic rays, and stellar flares before the search can be conducted.

While standard analysis tools already exist, handling large data efficiently remains a hurdle. Many baseline scripts run with a bottleneck by running heavy searches like phase-folding, which may fake the detection

To address this concept, this project adds a data processing pipeline using Python, using the LightKurve library, and numerical techniques. The objectives of this work include:

- **Data Acquisition & Filtering:** Requesting target Pixel Files (TPFs) directly from the Mikulski Archive for Space Telescopes (MAST) and adding a custom filter to remove the noise.
- **Efficient Period Search:** Running Box Least Squares (BLS) to scan for periodic transits and calculate orbital periods.
- **Efficient Period Search:** Running Box Least Squares (BLS) to scan for the periodic transits and calculate the orbital periods
- **False-Positive Elimination:** Adding checks such as odd or even depth evaluations to separate the planetary signals from the binaries and instrumental glitches

2 Methodology

METHODOLOGY

The pipeline receives the raw space telescope data by querying the Mikulski Archive for Space Telescopes (MAST) using LightKurve. Target Pixel Files

(TPFs) for specific targets are automatically pulled using their TIC IDs. Furthermore, because raw space measurements contain instrumental noise, background light, and stellar activity, a flattening filter is applied. This process makes the star's natural baseline instability, smoothing it to a flux of 1.0, while keeping its sharp transit signatures. To identify the repeating transit signals hidden in the data, the pipeline uses a Box Least Squares (BLS) spectrum. By phase-folding the light curve over multiple trial orbital periods, the faint dips are combined to maximize the signal-to-noise ratio (SNR) To rule out false candidates, the potential exoplanets undergo auto-vetting procedures. Specifically, an odd-even transit depth test that evaluates the alternating depth ratio to identify the exoplanet transits from eclipsing binary systems.

3 Data Script

Below is a Python script that catches and cleans TESS light kurve data using the LightKurve library, and numerical techniques. The objectives of this work include:

- **Data Acquisition & Filtering:** Requesting target Pixel Files (TPFs) directly from Mikulski Archive for Space Telescopes (MAST) and adding a filter to remove noise.
- **Efficient Period Search:** Running Box Least Squares (BLS) to scan for periodic transits and calculate orbital periods.
- **False-Positive Elimination:** Adding checks like odd or even depth evaluations to separate planet signals from instrumental glitches.

4 Pipeline Implementation

4.1 Python Data Ingestion Script

```
import lightkurve as lk
import numpy as np

# Step 1: Download the target pixel file using a TIC ID (e.g
    ., a known planet-hosting star)
search_result = lk.search_targetpixelfile("TIC␣25155310",
    mission="TESS")
tpf = search_result[0].download()

# Extract the raw light curve from the pixels
lc = tpf.to_lightcurve()

# Step 2: Clean and flatten the light curve (removes
    instrumental noise and baseline wiggles)
```

```
flat_lc = lc.flatten(window_length=402)

# Quick check: plot the flattened light curve
flat_lc.plot()

periodogram = flat_lc.to_periodogram(method="bls")  # Python
    scans through hundreds of trial periods behind the
    scenes, calculates the statistical power for each one,
    and builds a periodogram (a graph of periods vs. power).

best_period = periodogram.period_at_max_power
print(f"Best period: {best_period}")                    # It
    pulls out the single number (in days) that represents
    your candidate's orbital period.

folded_lc = flat_lc.fold(period=best_period)
folded_lc.plot()                                        #It
    takes a messy 27-day timeline and squishes it down into a
    single orbital cycle, forcing the faint transit dip to
    align cleanly in the center!

# Combine your time and flux arrays into a single table
output_data = np.column_stack((flat_lc.time.value, flat_lc.
    flux.value))

# Save it to a plain text file that C++ can read easily
np.savetxt("file.txt", output_data, delimiter=",", header="
    time,flux", comments="")
print("Data exported successfully for C++! :D")
```

5 C++ Data Processing Pipeline

Next, we feed this exported CSV data into a C++ program to perform the high-performance period-search.

```
#include <bits/stdc++.h>
#include <fstream>
#include <iostream>

int main()
{

    std::string filename = "file.txt";
    std::ifstream inFile(filename);    // combining
        declaration and open

    //SAFTEY CHECK
    if (!inFile.is_open())
```

```
    {
        std::cerr << "Error: Could not open the data file: "
            << filename << std::endl;
        return 1;
    }

    std::string line;

    std::string header;
    std::getline(inFile, header); // Eats and ignores the "
        time,flux" line


    std::vector<double> time;
    std::vector<double> flux;


    // 4. Efficient line-by-line loop
    while (std::getline(inFile, line))
    {
        std::stringstream ss(line);
        double t_val, f_val;
        char comma;

        // Read into  variabels instead of the vectors
            directly
        if (ss >> t_val >> comma >> f_val) {
            time.push_back(t_val);   // Add the time to our
                vector list
            flux.push_back(f_val);   // Add the flux to our
                vector list
        }
    }

    // 5. Clean up
    inFile.close();
    std::cout << "Successfully loaded " << time.size() << "
        data points at C++!" << "SUCCESS" << std::endl;


    double min_period = 0.5;     //min orbital time
    double max_period = 15.0;    //TESS looks at a exoplanet
         for 27 days. Half of it is roughly 15, which lets
        TESS see the orbital period twice
    double step_size = 0.001;    //  resolution of the
        period search, can be lower but it will take longer
        to compute

    std::vector<double> test_periods;   //create a empty
        vector to store the periods we will test
```

```
    for(double period = min_period; period <=max_period;
        period+= step_size)
    {
        test_periods.push_back(period);   //takes each of
            the trial generated by the loop and then staples
            it to the end
    }                                          // the loop
        creates the 0.5, 0.501, 0.502, 0.503, and so on all
        the way to 15. So it jst stores it
    std::cout << "Success!␣Loaded␣" << time.size() << "␣data
        ␣points␣from␣TESS!" << std::endl;
    return 0;
}
```

```
Best period: 3.291262582700361 d days
Data exported successfully as file.txt for C++!
Data exported successfully for c++
```

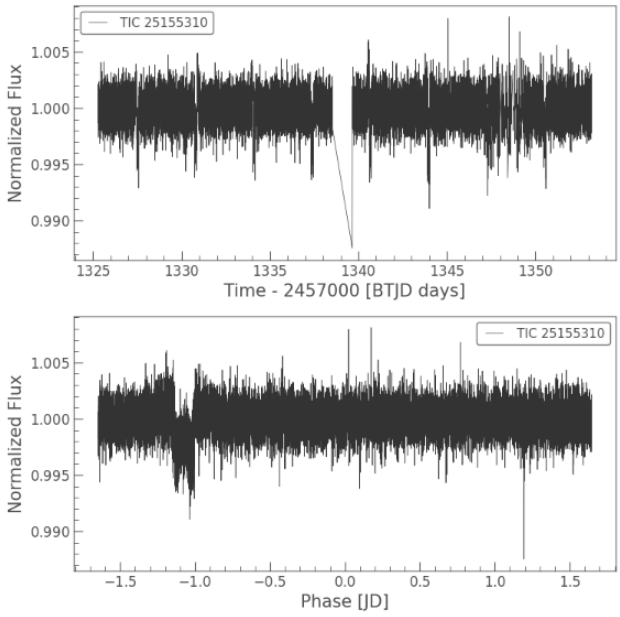


Figure 1: Flattened TESS light curve research for TIC 25155310.